

National University of Technology



Computer Science Department

Semester Spring – 2026

Program: Artificial intelligence

Course: NLP Lab

Course Code: CS387

Submitted To:

Lec. Rooshan Saleem

Submitted By:

Ahad Imran

F23607034

CiteGraph-NLP: Confidence-Aware Citation Lineage Analysis.....	2
Project Report.....	2
1. What Is This Project?.....	2
2. Problem Statement.....	2
3. System Architecture.....	3
Technology Stack.....	4
4. The Pipeline — Stage by Stage.....	5
Stage 1: Input Normalization.....	5
Stage 2: Metadata Resolution.....	5
Stage 3: Citation Traversal.....	6
Stage 4: Population Extraction.....	6
Extraction Method: Regex Pattern Matching.....	6
False Positive Prevention.....	7
Semantic Classification.....	7
Confidence Scoring.....	8
Sentence Splitting.....	8
Stage 5: Population Resolution.....	8
Resolution Algorithm.....	8
Stage 6: Edge Weighting & Graph Analytics.....	9
Edge Weighting Formula.....	9
Foundational Paper Ranking.....	9
Citation Path Ranking.....	9
5. The Frontend.....	10
Architecture.....	10
Key Pages.....	10
Data Flow.....	11
Citation Graph Visualization.....	11
6. Confidence-Aware Design Philosophy.....	11
7. Limitations & Honest Assessment.....	12
8. What Makes This an NLP Project.....	12
9. Example Walkthrough.....	13
10. Deployment Architecture.....	14
11. Academic Context.....	15

Group Members:

- M. Ahad Imran (F23607034)
- Syed Zain-ul-Abidin (F23607031)
- Hamza Abdul Karim (F23607046)
- M. Usman Nasir (F23607004)

CiteGraph-NLP: Confidence-Aware Citation Lineage Analysis

Project Report

1. What Is This Project?

CiteGraph-NLP is a system that takes a single research paper (identified by DOI, PMID, or title) and automatically:

1. Resolves its metadata from multiple scholarly APIs
2. Traverses its citation network backward and forward
3. Extracts study population sizes from paper abstracts using NLP
4. Builds a weighted citation knowledge graph
5. Ranks the most probable **foundational papers** behind the research idea

The core insight is that **citation count alone does not tell you where a research idea came from**. A paper with 5,000 citations might be a popular review, while the actual foundational study with the original randomized trial sits three citation hops away with only 200 citations. CiteGraph-NLP uses population-size evidence and confidence-aware edge weighting to surface papers that matter structurally, not just numerically.

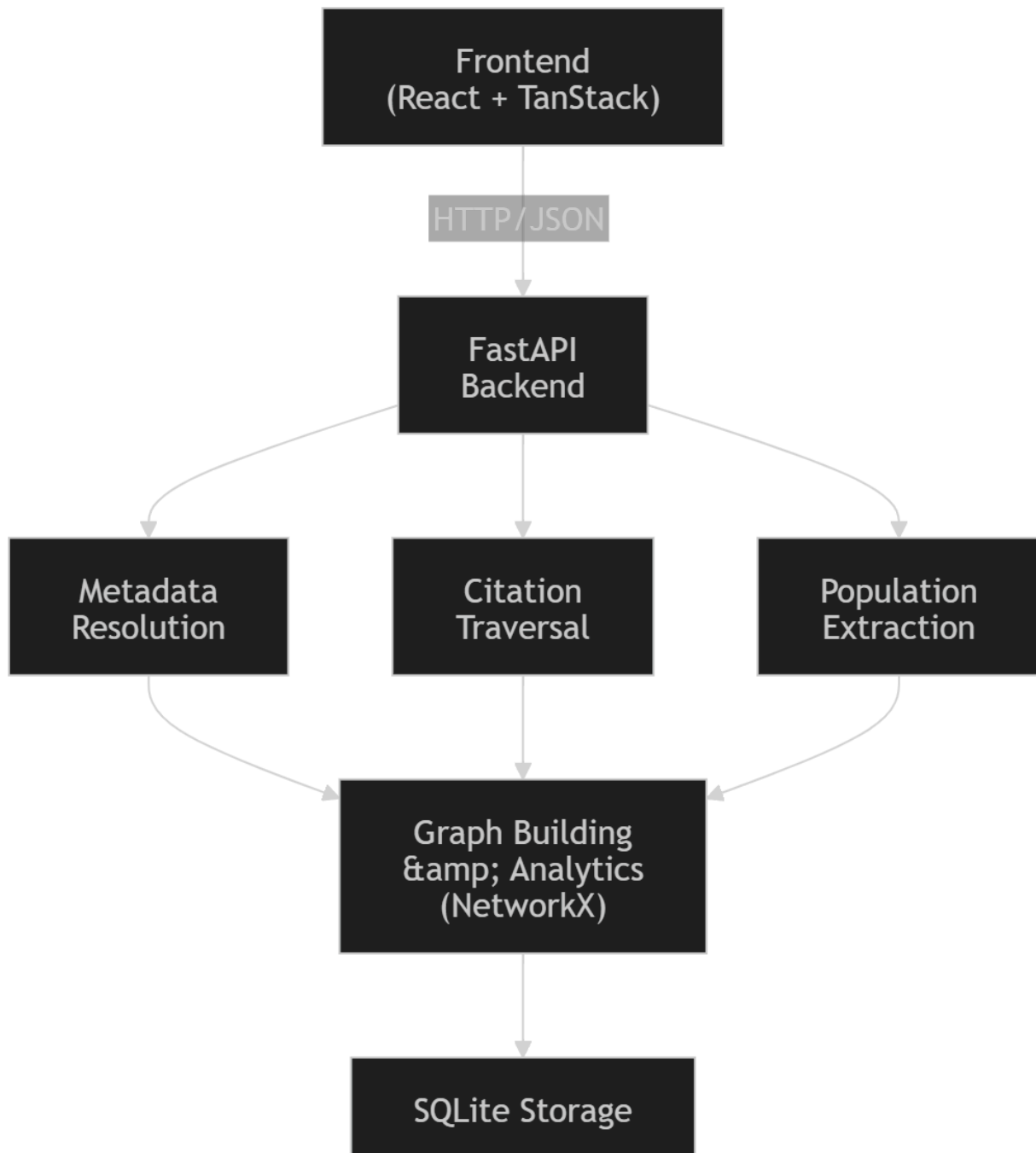
2. Problem Statement

Modern research papers cite dozens of earlier works. Those papers cite even more. Important questions arise:

- **Which earlier paper is the actual foundation?** The most-cited paper is not necessarily the most foundational.
- **What is the study-scale evidence behind each paper?** A paper claiming "8,500 patients were randomized" carries different weight than one with "n=24."
- **How confident are we in each piece of extracted data?** Not every number in a paper is a study population. Extraction is inherently uncertain.

Manual literature tracing is slow, incomplete, and hard to reproduce. CiteGraph-NLP automates this with a structured, confidence-aware pipeline.

3. System Architecture



Technology Stack

Layer	Technology
Frontend	React 19, TypeScript, TanStack Router/Query, Tailwind CSS 4
Backend	Python 3.10+, FastAPI, Pydantic v2
NLP	spaCy (sentence splitting), regex pattern matching
Graph	NetworkX (in-memory graph algorithms)
Storage	SQLite via aiosqlite (async)
Metadata Providers	OpenAlex, Crossref, Europe PMC
Deployment	Docker (backend), Firebase Hosting (frontend)

4. The Pipeline — Stage by Stage

When a user submits a paper identifier, the system executes six sequential stages:

Stage 1: Input Normalization

The system accepts multiple identifier formats:

Input Type	Example
DOI	<code>10.1056/NEJMoa2034577</code>
PMID	<code>33882225</code>
PMCID	<code>PMC8012345</code>
Title	<code>Attention Is All You Need</code>
URL	<code>https://pubmed.ncbi.nlm.nih.gov/33882225/</code>

The **IdCanonicalizer** normalizes all identifiers into canonical forms. DOIs are lowercased, PMCID IDs are uppercased, URLs are parsed to extract embedded identifiers, and OpenAlex IDs (W-prefixed) are preserved.

Stage 2: Metadata Resolution

The **MetadataResolver** queries all enabled providers **in parallel** using `asyncio.gather()`:

- **OpenAlex** — Comprehensive metadata, abstract reconstruction from inverted index, reference/citation data
- **Crossref** — Authoritative DOI metadata, publication dates, author lists
- **Europe PMC** — Strong for biomedical papers, full-text abstracts

The **MetadataMerger** then combines results using field-level precedence rules:

title, year, authors, journal → prefer Crossref, then OpenAlex

abstract → prefer OpenAlex (reconstructed from inverted index)

Metadata confidence is assigned based on provider agreement:

- Multiple providers agree: **0.95**
- Single provider: **0.80–0.90** (provider-dependent)

Each paper also stores a **provenance** dictionary tracking which provider contributed which fields, preserving full data lineage.

Stage 3: Citation Traversal

The **CitationTraversal** class performs **breadth-first search (BFS)** through the citation network in two directions:

Backward traversal follows a paper's references (papers it cites). With `backward_depth=2`, the system traces two hops: the seed paper's references, then those papers' references.

Forward traversal follows citations (papers that cite a given paper). With `forward_depth=1`, only direct citations of the seed are included.

Key design decisions:

- **Deduplication**: Edges are deduplicated by `(source_id, target_id)` pairs. If multiple providers report the same citation, the `providers` list is merged.

- **Hard paper limit** (`max_papers=100`): Prevents memory explosion; BFS stops when the limit is reached.
- **Forward depth is conservative** (default 1): Forward citation graphs grow exponentially; deeper traversal would overwhelm the system.

The citation retriever queries all providers in parallel for each paper, then merges and deduplicates the results.

Stage 4: Population Extraction

This is the NLP core of the system. The `PopulationExtractor` processes each paper's abstract to find candidate study population sizes.

Extraction Method: Regex Pattern Matching

The system uses a library of compiled regex patterns, each targeting a specific way population sizes appear in biomedical text:

Pattern	Example Match	Semantic Type	Base Confidence
<code>(\d+)\s+patients\s+were\s+randomized</code>	"8,500 patients were randomized"	TOTAL_RANDOMIZED	0.95
<code>enrolled\s+(\d+)</code>	"enrolled 5,200"	TOTAL_ENROLLED	0.85
<code>analyzed\s+(\d+)</code>	"analyzed 3,100"	TOTAL_ANALYZED	0.85
<code>N\s*=\s*(\d+)</code>	"N = 240"	SAMPLE_SIZE_GENERIC	0.80
<code>cohort\s+of\s+(\d+)</code>	"cohort of 1,200"	SAMPLE_SIZE_GENERIC	0.75
<code>(\d+)\s*patients</code>	"1,121 patients"	TOTAL_ENROLLED	0.70

False Positive Prevention

Not every number in a paper is a population size. The system applies **ignore patterns** to filter out:

- Years (2020, 2021)
- Percentages (45%)
- P-values ($p < 0.05$)
- Confidence intervals (95% CI)

- Dosages (200 mg)

A **sanity check** also rejects values ≤ 0 or $> 10,000,000$.

Semantic Classification

Each extracted value is classified into one of these types:

TOTAL_RANDOMIZED — All randomized participants (gold standard)

TOTAL_ANALYZED — Analyzed population (post-dropout)

TOTAL_ENROLLED — Enrolled before randomization

ARM_SIZE — Per-arm sample size

SCREENED — Screened for eligibility

COMPLETERS — Completed the full protocol

SAMPLE_SIZE_GENERIC — Non-specific sample size

UNKNOWN_NUMERIC — Unclassified

Confidence Scoring

confidence = base_pattern_weight + section_bonus

- **Base weight** (0.70–0.95): Determined by the matched pattern. More specific patterns (e.g., "X patients were randomized") get higher base weights.
- **Section bonus** (+0.10): If the match comes from the abstract or methods section, confidence increases.
- **Final confidence**: Clamped to `[0.0, 1.0]`.

Sentence Splitting

If spaCy is available, the system uses spaCy's sentence tokenizer for accurate sentence boundary detection. Otherwise, it falls back to a regex-based splitter `((?<=[.!?])\s+)`.

Stage 5: Population Resolution

After extraction, each paper may have **multiple candidate population values**. The `PopulationResolver` selects the single best estimate (`n_eff`) for each paper.

Resolution Algorithm

1. **Filter** candidates below a minimum confidence threshold (0.4)

2. **Score** each candidate: $\text{score} = \text{confidence} \times (1 + \text{type_priority} / 10)$
 - Type priority: $\text{TOTAL_RANDOMIZED} (10) > \text{TOTAL_ANALYZED} (9) > \text{TOTAL_ENROLLED} (8) > \dots > \text{UNKNOWN_NUMERIC} (0)$
3. **Select** the highest-scoring candidate as n_{eff}
4. **Detect ambiguity**: If another candidate scores within 90% of the best but differs in value by more than 10%, mark the resolution as "ambiguous" instead of "resolved"

This ensures that:

- TOTAL_RANDOMIZED values are preferred over generic counts
- High-confidence extractions win over low-confidence ones
- Genuine ambiguity is preserved rather than hidden

Stage 6: Edge Weighting & Graph Analytics

Edge Weighting Formula

For each citation edge from paper A $\rightarrow$ paper B:

$$n_{\text{score}} = \log(1 + B.n_{\text{eff}}) / \log(1 + 100,000) \quad [0.0 - 1.0]$$

$$\text{journal_score} = 0.5 \quad [\text{neutral in MVP}]$$

$$\text{base_weight} = (0.75 \times n_{\text{score}}) + (0.25 \times \text{journal_score})$$

$$\text{final_weight} = \text{base_weight} \times B.\text{pop_confidence} \times \text{edge.confidence}$$

The key insight: **edges pointing to papers with larger, more reliable populations get higher weights**. This biases the graph toward evidence-rich citation paths.

Foundational Paper Ranking

The system combines three signals using NetworkX:

$$\text{influence_score} = \text{PageRank}(\text{graph}, \text{weight}='final_weight') \quad [50\%]$$

$$\text{year_score} = \min(1.0, \text{years_old} / 20) \quad [30\%]$$

$$\text{evidence_score} = n_{\text{score}} \times \text{population_confidence} \quad [20\%]$$

$$\text{foundational_score} = 0.5 \times \text{influence} + 0.3 \times \text{year} + 0.2 \times \text{evidence}$$

- **Influence (50%)**: Weighted PageRank captures structural importance. Papers cited by strong-evidence papers inherit that strength.

- **Age (30%)**: Older papers are more likely foundational. A 2002 trial is more foundational than a 2020 meta-analysis.
- **Evidence (20%)**: Papers with larger, confidently-extracted populations are more credible.

This creates a ranking that is fundamentally different from citation count: a 1989 paper with 166 randomized patients can outrank a 2016 paper with 195 million (a global estimate, not a study population) because the algorithm accounts for semantic type, confidence, and graph position.

Citation Path Ranking

The system also finds and ranks **citation chains** from the seed paper to deeper papers:

For each reachable paper within 4 hops:

Find all simple paths from seed to that paper

$\text{path_score} = \text{avg}(\text{edge_weights}) \times \text{product}(\text{edge_confidences}) \times \text{depth_bonus}$

$\text{depth_bonus} = 1 / \text{sqrt}(\text{path_length})$

Shorter, higher-weight paths rank first — representing the most direct, evidence-rich lineage connections.

5. The Frontend

Architecture

The frontend is a single-page application built with:

- **React 19** with TypeScript
- **TanStack Router** for file-based routing
- **TanStack Query** for data fetching with automatic polling
- **Tailwind CSS 4** with a custom dark theme
- **Vite** for build tooling

Key Pages

Route	Component	Purpose
/	Landing page	Marketing/information

Route	Component	Purpose
<code>/start</code>	Start form	Submit DOI/PMID/title to begin analysis
<code>/dashboard/:runId</code>	Overview	Summary stats, pipeline progress, warnings
<code>/metadata/:runId</code>	Metadata table	Searchable/sortable paper table with detail drawer
<code>/population/:runId</code>	Population page	Extraction results, candidate evidence, confidence meters
<code>/graph/:runId</code>	Citation graph	Interactive SVG visualization with pan/zoom/drag
<code>/paths/:runId</code>	Citation paths	Ranked evidence chains between papers
<code>/rankings/:runId</code>	Rankings	Foundational paper rankings with scores
<code>/export/:runId</code>	Export	Download JSON, CSV, GraphML, Markdown

Data Flow

1. User submits a paper identifier on `/start`
2. Frontend POSTs to `/api/runs` and receives a `run_id`
3. Frontend redirects to `/dashboard/:runId`
4. TanStack Query polls `GET /api/runs/:runId` every 2 seconds
5. When `status === "completed"`, all pages render with the full `RunResult`

Citation Graph Visualization

The graph is a custom SVG implementation (no external graph library):

- **Node size** = effective population (N_{eff}), log-scaled
- **Node color** = confidence level (green/amber/red/gray)
- **Edge thickness** = final citation weight
- **Edge direction** = citation lineage (later $\rightarrow$ earlier)
- **Purple ring** = foundational candidate
- **Pulsing ring** = seed paper
- Supports pan, zoom, drag, click-to-select, and hover tooltips

6. Confidence-Aware Design Philosophy

The defining feature of CiteGraph-NLP is that **uncertainty is made visible at every stage**, rather than hidden:

Stage	Uncertainty Signal	How It's Exposed
Metadata	Provider disagreement	<code>metadata_confidence</code> score, <code>provenance</code> dict
Population extraction	Pattern ambiguity	Multiple candidates shown, confidence per candidate
Population resolution	Competing values	<code>"ambiguous"</code> status instead of forced selection
Citation edges	Provider coverage	<code>providers</code> list, edge <code>confidence</code>
Edge weights	Propagated confidence	<code>final_weight</code> incorporates upstream confidence
Rankings	Combined uncertainty	Score breakdown (influence + age + evidence)

This means the system never pretends it found "the" foundational paper with certainty. It says: "Here are the most probable foundational papers, ranked by citation influence, age, and population evidence quality — with confidence scores so you can judge."

7. Limitations & Honest Assessment

1. **No full-text extraction:** Only abstracts are processed. Methods sections (which contain the richest population data) are only accessible via GROBID/PDF parsing, which is optional infrastructure.
2. **Regex-based NLP:** The population extractor uses pattern matching, not machine learning classifiers. This misses unusual phrasings and non-English text.
3. **Citation database gaps:** No scholarly database has complete citation coverage. OpenAlex is the most comprehensive but still misses some edges.

4. **No cross-study linking:** Population resolutions are per-paper. The system doesn't link the same clinical trial across multiple papers.
 5. **Depth limits:** Default backward depth of 2 means papers more than 2 citation hops from the seed are invisible.
 6. **No guarantee of completeness:** The system identifies *probable* foundational papers, not *definitive* ones. The actual original paper may not be in any citation database.
-

8. What Makes This an NLP Project

CiteGraph-NLP integrates several NLP and information extraction techniques:

1. **Named numeric extraction** — Identifying population-size numbers in unstructured text using pattern-based NER
2. **Semantic classification** — Categorizing extracted values into semantic types (TOTAL_RANDOMIZED, ARM_SIZE, etc.)
3. **Sentence boundary detection** — Using spaCy's statistical sentence tokenizer
4. **Section-aware scoring** — Boosting confidence when evidence appears in expected sections (Methods, Abstract)
5. **Ambiguity detection** — Identifying cases where multiple plausible values exist and explicitly marking uncertainty
6. **Evidence sentence extraction** — Capturing the exact sentence containing each population value for human review
7. **False positive filtering** — Rejecting years, p-values, dosages, and other numeric confounders

The NLP output directly feeds into the graph analytics: population evidence is the primary signal in edge weighting, which in turn drives PageRank-based foundational paper ranking.

9. Example Walkthrough

Input: DOI [10.1056/NEJMoa1402869](#) (a hepatitis C multicenter randomized trial)

Stage 1: Normalized to DOI [10.1056/nejmoa1402869](#)

Stage 2: OpenAlex and Crossref both return metadata. Merged with confidence 0.96. Title: "Sustained virologic response in chronic hepatitis C: a multicenter randomized trial"

Stage 3: BFS finds 47 papers across 2 backward hops and 1 forward hop, with 126 citation edges.

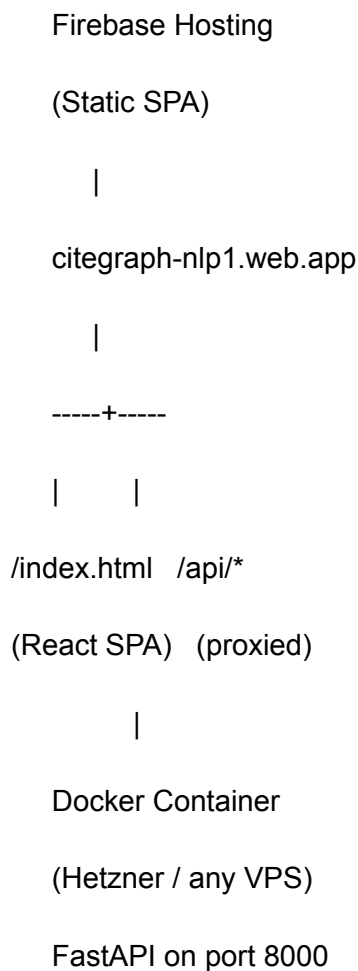
Stage 4: From the seed paper's abstract, the extractor finds:

- Candidate 1: 8,500 from "8,500 patients were randomized" → TOTAL_RANDOMIZED, confidence 0.92
- Candidate 2: 7,984 from "7,984 patients completed" → COMPLETERS, confidence 0.74

Stage 5: Resolution selects $n_{eff} = 8,500$ (higher confidence, higher-priority semantic type). Status: "resolved".

Stage 6: Edge weights computed. A 1989 interferon trial (166 patients, but structurally central via PageRank) ranks as the #1 probable foundational paper, ahead of a 2016 global burden paper (195 million, but population-level estimate with low confidence).

10. Deployment Architecture



|

SQLite + NetworkX

(local file storage)

- Frontend is a static build hosted on Firebase
 - Backend runs as a Docker container on any Linux VPS
 - No managed database required — SQLite handles persistence
 - The `VITE_API_BASE` environment variable connects frontend to backend
-

11. Academic Context

This project sits at the intersection of:

- **Natural Language Processing** — Information extraction from scientific text
- **Knowledge Graphs** — Representing scholarly relationships as typed, weighted graph structures
- **Bibliometrics** — Quantitative analysis of citation patterns
- **Information Retrieval** — Querying and aggregating metadata from multiple scholarly APIs

It demonstrates a realistic NLP pipeline: messy real-world input → structured extraction → uncertainty quantification → downstream graph analytics → actionable output.